

UST-2026-09-26-18

Incidente waitlist: 630.083 correos sintéticos entraron en bloque por el endpoint público (10 y 11-sep) y dejaron el  Piloto en blanco 16 días — limpieza, guardas, forense y borrado

AS-BUILT · DECIDIDA y FIRMADA por Juanjo (26-sep: «UST-18, según las recomendaciones también»): D1-D6 y D9 tal como se proponen; D7 con SU decisión previa (WL-1: «Limpiarlos, no se hace análisis, borrarlos» — ejecutada en el cierre r27-b, la cuarentena ya no existe) y D8 en consecuencia (sin análisis de Logs). Medido, limpiado y blindado el mismo día en carril técnico; Pilar y Lucía corroboran.

1 · El pedido, literal

Pilar, 21-sep (captura del  Piloto: «canceling statement due to statement timeout» dentro de la caja «sin acceso») · Juanjo, 26-sep («el Piloto no carga»; por la noche, sobre la cuarentena: «Limpiarlos, no se hace análisis, borrarlos»; y «¿cómo se puede prevenir que un bot meta esos correos falsos?» → UST-19):

«canceling statement due to statement timeout» (captura de Pilar, 21-sep, pintado por el hub como «sin acceso»)

«Limpiarlos, no se hace análisis, borrarlos» (Juanjo, 26-sep, sobre las 630.075 filas de la cuarentena — decisión WL-1)

Lectura: dos síntomas («no me deja entrar», «el Piloto no carga») que parecían de acceso y eran de TAMAÑO: la tabla `waitlist` había pasado de ~160 filas a 630.243 y las RPC del Piloto se pasaban de los 8 segundos que PostgREST concede a una consulta. La primera hipótesis del asistente («157 MB con ~160 filas → blobs o churn») era falsa: el recuento de la sonda de la mañana llevaba el filtro de la RPC y tapaba las filas basura.

2 · Lo que había (medido el 26-sep, antes de tocar)

Qué	Medido
Tamaño real	630.243 filas en `waitlist` (160 anteriores al 8-sep + 8 altas web2 reales de septiembre + 630.075 basura). 157 MB de tabla; el dump cifrado pasó de 3,1 a 23 MB y el del 25-sep falló por timeout. La web pintaba «630220+» en la Home.
Dos ventanas, un patrón	10-sep 15:13:33-15:59:21 UTC: 566.075 filas en 46 min (~205/s, 564.567 instantes distintos), correos con formato válido. 11-sep 13:28:42-13:51:49 UTC: 64.000 filas en 23 min, 19-22 caracteres SIN «@». Todas con `source = legacy_import` (el DEFAULT de la columna: nadie lo eligió).
Cómo entraron	`pg_stat_statements`: 630.076 llamadas del rol `anon` a `INSERT INTO waitlist(email)` vía PostgREST — una fila por petición, solo el campo `email`. Es decir: el endpoint público REST, con la anon key (pública por diseño), sin pasar por ningún formulario.
Qué eran	Correos sintéticos: NUTRIPOOP.com 371.022 · NUTRIbruh.com 191.053 · PLEASE-EMAIL-ME-BACK.com 4.000 (partes locales numéricas) · 64.000 cadenas sin «@». 0 cruces con `auth.users`, `pilot_invites` o la lista viva → NO había datos personales.
Consecuencias	`admin_waitlist` y `admin_pilot_overview` > 8 s →  Piloto en blanco desde el 10-sep (16 días); pestaña  lentísima; Home con cifra falsa; dumps hinchados. Nadie lo vio hasta la captura de Pilar del 21-sep, y esa captura estaba etiquetada como «sin acceso».
Por qué pudo pasar	La policy `anyone can join waitlist` (anon/authenticated, INSERT, `with check (true)`) era correcta FILA A FILA y ciega al VOLUMEN: ni tope de lote, ni ritmo, ni formato, ni origen obligatorio. Y nada medía el crecimiento diario de la tabla.

Analogía: la waitlist era un puerto abierto directamente al servidor de base de datos con una ACL por fila (RLS): cada paquete pasaba la regla, y la regla no sabía contar. Un API pública se protege con un gateway delante (rate limit, WAF, reto) y el puerto trasero cerrado — eso es UST-19. Lo de hoy son las guardas dentro del servidor: paran esta forma exacta de ataque y limitan cualquier otra.

3 · Lo que se hizo, fila a fila (as-built, 26-sep)

Qué	Qué se construyó	Cómo se verificó
C1 · Sonda de tamaño	`2026-09-26-sonda-waitlist-tamano.sql`: `count(*)` desnudo, por día y por `source`, últimas altas enmascaradas. Sustituye a la lectura de la mañana («157 MB con ~160 filas»).	Resultado pegado por Juanjo: 630.243 · 160 antes del 8-sep · 10-sep 566.077 · 11-sep 64.000 · legacy_import 630.215.
C2 · Limpieza 1 (cuarentena + guardas básicas)	`2026-09-26-waitlist-limpieza.sql`: tabla `waitlist_cuarentena_20260926` (RLS activada SIN políticas: solo el rol de base de datos) → movimiento del lote → índice `lower(email)` → CHECK `waitlist_email_formato` → trigger `waitlist_tope_lote` (>20 filas por sentencia desde anon/authenticated = rechazo; regla pura probada en rojo y verde).	Verificación en producción: vivas 566.243 · cuarentena 64.000 → el predicado «sin @» solo describía el lote del 11-sep. Lección: el predicado destructivo se comprueba por RECuento en el mismo fichero.
C3 · Compactar	`2026-09-26-waitlist-compactar.sql`: `CLUSTER public.waitlist USING <índice de id>` + `ANALYZE` (VACUUM FULL no cabe en la plantilla; CLUSTER reescribe la tabla igual y sí corre en transacción). Sustituye a `vacuum.sql`, que nunca se ejecutó y puso en rojo `sql-columnas`.	1ª pasada: heap 53 MB (aún con el lote del 10-sep). 2ª pasada tras la limpieza 2: heap 24 kB · 112 kB con índices · 168 vivas · cuarentena 90 MB.
C4 · Limpieza 2 (el lote del 10-sep + guardas de origen y ritmo)	`2026-09-26-waitlist-limpieza-2.sql`, idempotente, con `begin/commit` explícitos: aserción de recuento (566.075 con el MISMO predicado que mueve) → movimiento → `waitlist_anon_guard` (BEFORE INSERT: anon debe enviar `source` ∈ {web2, marketing_site, newsletter}; `created_at` := now(); `converted_at` := null) → `waitlist_tope_ritmo` (>200 altas en 10 min = rechazo; contador security definer) → reglas puras en rojo/verde → pruebas e2e como `anon` (set local role anon, como PostgREST) → forense sin PII → tablero + □ WL-1.	Cuatro intentos reales, cada uno una lección: (1) 566.077 ≠ 566.075 → abortó sin tocar (las 2 altas web2 del mismo día); (2) `min(uuid)` no existe → reventó en el forense, PERO el editor había aplicado los bloques 0-3 (autocommit por sentencia); (3) 0 vivas → abortó; (4) completo. Resultado final: 168 vivas · 630.075 en cuarentena · `waitlist_count()` 168.
C5 · Prueba de la guarda en producción	Juanjo, con curl: POST anónimo a `/rest/v1/waitlist` con solo `{ "email": ... }` (la forma exacta del ataque).	`HTTP/2 400` · `error=23514` · «waitlist: origen legacy_import no admitido en un alta pública». Las altas legítimas (web2, marketing_site, newsletter) siguen pasando: las pruebas e2e como anon lo ejercitaron en producción.
C6 · Arnés PostgreSQL 16 real	`tools/sql-arnes/` (embedded-postgres + pg en el sandbox): reproduce el editor de Supabase (sentencia a sentencia, autocommit), semilla con los NÚMEROS reales y los TIPOS de `information_schema` (id es uuid; el ancla decía bigint). Escenarios: fallo inyectado antes del commit, estado parcial del 2º intento, fichero real sobre el estado parcial, idempotencia.	Reprodujo EXACTAMENTE el 566.243/64.000 de producción antes de arreglar la limpieza 2, y la segunda pasada del cierre r27-b aborta por aserción sin tocar nada.
C7 · Deploy Web más seguro (de rebote)	`sql-columnas` corre ANTES del unzip; toda puerta posterior restaura `publish/` al último commit al ponerse en rojo (`abortar()`). El rojo de `vacuum.sql` había dejado 103 ficheros sin commitear en webdeploy.	Probado en rojo y en verde sobre el árbol real; po254 salió por ese camino.
C8 · Decisión WL-1 y borrado	Cierre r27-b (`2026-09-26-cierre-r27b-tablero.sql`): WL-1 aceptada con la decisión literal de Juanjo; `drop table waitlist_cuarentena_20260926` SOLO si la tabla tiene exactamente 630.075 filas dentro de las dos ventanas y 0 correos cruzan con cuentas, piloto o lista viva. Si no cuadra, no borra y el fichero entero se deshace.	Arnés: 5 escenarios en verde (rollback por fallo inyectado · recuento ≠ 630.075 → aborta · un correo real colado → aborta por la guarda anti-PII · real · idempotente). En producción: pendiente de Run (esta misma tanda).
C9 · Hub y web	Sin cambios de código: `admin_waitlist`/`admin_pilot_overview` vuelven a bajar de los 8 s por tamaño; `waitlist_count()` pinta «160+» (168 redondeado a la decena).	Verificación humana pendiente (Pilar y Lucía abren el Piloto). El caso «una tabla crece y tumba el hub» se cierra de raíz con RPCs paginadas (item S19).

4 · Análisis de impacto

Dónde	Qué toca	Qué NO toca
Base de datos · waitlist	Filas: -630.075 (sintéticas). Nuevo: índices `lower(email)` y `created_at`, CHECK de formato, 3 triggers (tope de lote, guarda de origen para anon, tope de ritmo), 5 funciones puras/auxiliares. La tabla queda CLUSTER-izada (24 kB).	Las 168 filas reales (160 + 8 de septiembre): ninguna se toca. Las polícies RLS (siguen las mismas — `anyone can join waitlist` se retira en UST-19).
Base de datos · cuarentena	Tabla `waitlist_cuarentena_20260926` creada (RLS sin políticas) y BORRADA en el cierre r27-b (WL-1).	El dump cifrado de R2 (cada 4 h, 30 días) anterior al borrado la conserva por si alguien quisiera mirarla: nadie quiere.
Formularios públicos	Ninguno cambia: web2 envía `source = web2`, el marketing viejo `marketing_site`, el pie `newsletter` → la guarda de origen los deja pasar. Una petición «desnuda» (solo email) ya no entra.	La app nativa y la PWA no escriben en `waitlist`.
Hub	☐ Piloto y ☐ Waitlist vuelven a cargar. Ningún cambio de código en esta UST.	Las RPC (misma firma). La paginación es item aparte (S19).
Tooling	`tools/Deploy NutriSync Web.command` (orden de puertas + `abortar()`), `tools/sql-arnes/` (nuevo).	OTA, builds, PWA.
Quién lo recibe	Las founders (hub sano, cifra real en la Home). Las ~168 personas de la lista: nada cambia para ellas.	Testers del piloto (tabla distinta).

Origen: ABUSO EXTERNO del endpoint público, no un fallo nuestro de datos ni una importación. Pendiente y ya no necesario tras WL-1: IP y user-agent en Supabase → Logs → API (POST /rest/v1/waitlist, esas dos ventanas). Deuda estructural: el alta pública sigue siendo un INSERT directo con la anon key → UST-19.

5 · Decisiones (Juanjo, 26-sep, carril técnico — no hay datos de usuarias en juego)

#	Propuesta	Alternativa	Decisión
D1	Cuarentena con RLS sin políticas en vez de borrar directo: el origen no se conocía y la marcha atrás es un `insert ... select`.	Borrar directo.	DECIDIDO: la propuesta (cuarentena) — y borrada el 26-sep por WL-1 una vez medido que no había PII.
D2	Tope de LOTE en el servidor: >20 filas por sentencia desde anon/authenticated = rechazo (trigger por sentencia con tabla de transición).	Confiar en RLS + rate limit del proveedor.	DECIDIDO: la propuesta.
D3	Origen OBLIGATORIO para anon (`source` ∈ {web2, marketing_site, newsletter}); `created_at` lo pone el servidor.	Solo CHECK de formato.	DECIDIDO: la propuesta.
D4	Tope de RITMO global: 200 altas / 10 min (contador security definer). Por IP no es fiable desde un trigger → UST-19 (Edge).	Sin tope de ritmo.	DECIDIDO: la propuesta (global); el ritmo por IP va a UST-19.
D5	CLUSTER en su propia pasada, DESPUÉS del commit del borrado (medido: en la misma transacción no libera nada — 46 MB antes y después en el arnés).	VACUUM FULL (no cabe en la plantilla ni en la bandeja).	DECIDIDO: la propuesta (CLUSTER, pasada propia).
D6	Forense SIN datos personales: dominios, formas, instantes, cruces por recuento; muestras enmascaradas (`p***@dominio`).	Volcar correos completos al chat/tablero.	DECIDIDO: la propuesta.
D7	WL-1 · Qué hacer con las 630.075 filas: recomendación de ingeniería «conservar 7 días para el forense y borrar».	Borrar ya · Conservar hasta saber el origen.	DECIDIDO (Juanjo, 26-sep): ALTERNATIVA «Borrar ya» — «Limpiarlos, no se hace análisis, borrarlos». Sin PII, es carril técnico; Pilar y Lucía corroboran.
D8	IP y user-agent en los Logs de Supabase de esas dos ventanas (forense externo).	No hacerlo.	DECIDIDO: NO se hace (WL-1: sin análisis).

#	Propuesta	Alternativa	Decisión
D9	Todo fichero que mueva datos en masa pasa por el arnés PG16 antes de la bandeja, con aserciones de recuento y `begin/commit` explícitos.	Parse + sql-columns (lo que había: dio verde a los dos ficheros que fallaron).	DECIDIDO: la propuesta — regla nueva en CLAUDE.md.

6 · Baseline, verificación y marcha atrás

Momento	Qué se comprueba
PASO 0 · Baseline (antes de tocar)	Sondas de solo lectura `sonda-recalibracion-s19.sql` y `sonda-waitlist-tamano.sql` (recuentos por día y fuente) + `baseline-20260926-191427.txt`. Dump cifrado de R2 de las 4 h anteriores (30 días).
Limpieza 1 (≈21:00)	vivas 566.243 · cuarentena 64.000 · índice/CHECK/trigger true · `waitlist_count()` 566.242 → parcial, como se explica en C2.
Compactar 1	heap 53 MB · 140 MB total · 566.243 vivas (el lote del 10-sep seguía).
Limpieza 2 (4 intentos, ≈22:0x-22:2x)	Sección A: vivas 168 · cuarentena 630.075 · `waitlist_count()` 168. Sección B: guardas creadas y probadas como anon. Secciones C-E: forense (instantes, dominios, 0 cruces, 630.076 llamadas anon en pg_stat_statements). Sección F: tablero + WL-1.
Compactar 2	heap 24 kB · 112 kB con índices · 168 vivas · cuarentena 90 MB.
Guarda en vivo (Juanjo, curl)	`HTTP/2 400` · `proxy-status: PostgREST; error=23514` · «waitlist: origen legacy_import no admitido en un alta pública».
Cierre r27-b (pendiente de Run)	WL-1 aceptada · cuarentena existe = false · vivas 168 (+ altas reales nuevas) · arnés 5/5.
Marcha atrás	Guardas: `drop trigger` de los tres + `drop function` (nombres en el fichero). Filas: hasta el borrado, `insert into waitlist select * from waitlist_cuarentena_20260926`; después, el dump de R2. Deploy Web: `git checkout` del .command. No se quiere ninguna de las tres.

7 · Cómo lo comprueban Pilar y Lucía

- Piloto carga en unos segundos (antes: en blanco o «sin acceso»).
- Waitlist muestra 168 personas (160 de siempre + 8 apuntadas en septiembre) y ninguna cadena rara sin «@».
- La Home de nutrisynccollective.com dice «160+» (antes «630220+»).
- El formulario de la lista de espera de la web sigue funcionando: apuntarse con un correo propio → aparece en □ en el acto.
- Nadie de la lista real ha desaparecido (las 168 son las de siempre).

8 · Lecciones grabadas (van también a CLAUDE.md)

Lección	Qué cambia
«Sin acceso» no era acceso	Todo mensaje de error de una RPC viaja LITERAL a la pantalla (r12-b9); la etiqueta «sin acceso» escondió un «statement timeout» 16 días. Item S20: nsFail pinta el error real.
Un tamaño que no cuadra se cuenta desnudo	`count(*)` sobre la tabla y agrupado por día/fuente ANTES de teorizar sobre blobs o churn. El recuento con el filtro de la RPC tapaba 630.083 filas.
RLS no sabe contar	Una tabla pública con insert anónimo necesita tope de LOTE, origen obligatorio y ritmo — y aun así el gateway va delante (UST-19).
El predicado destructivo se comprueba por recuento en el mismo fichero	Y con el MISMO predicado que borra: 566.077 (por día) ≠ 566.075 (legacy_import) porque hubo 2 altas reales ese día. Aborta entero si no cuadra.
El editor de Supabase autocommitea sentencia a sentencia	Medido en el 2º intento: un fallo a mitad deja aplicado lo anterior. Todo fichero con más de una cosa lleva `begin;` tras la cabecera y `commit;` antes de la verificación; los bloques son idempotentes; ante «current transaction is aborted», `rollback;` a solas.

Lección	Qué cambia
CLUSTER/VACUUM FULL en la misma transacción que el DELETE no libera nada	Compactar va en su propia pasada, después del commit (46 MB antes y después en el arnés).
El SQL se prueba ejecutándolo	Parse PG16 + sql-columnas dieron verde a dos ficheros que fallaron. Ahora hay un PostgreSQL 16 real en el sandbox y todo fichero que mueva datos en masa pasa por él, con la semilla copiada de la BASE (tipos de information_schema: id es uuid).
Un detector se lee ENTERO	Leí `tail -1` de sql-columnas y lo llamé verde; la cabecera « X 2 problema(s)» estaba dos líneas arriba y abortó el Deploy Web de Juanjo.
Una puerta que corre después de escribir deja el árbol sucio	sql-columnas va antes del unzip; toda puerta posterior restaura publish/ al ponerse en rojo (`abortar()`).
Antes de pedir una excepción a una puerta, buscar la forma del cambio que la cumple	VACUUM no cabía en la plantilla; CLUSTER hace lo mismo y sí cabe.

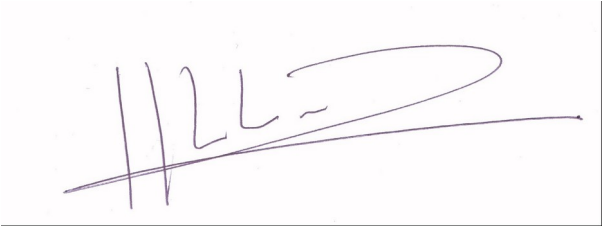
9 · La cadena

Paso	Estado
UST	Este documento (as-built del 26-sep). Se escribe el mismo día, con lo construido y sus desviaciones.
Sign-off	Carril técnico (sin datos de usuarias): Juanjo decidió en el chat, incluida WL-1 («borrarlos»), y FIRMÓ la noche del 26-sep («UST-18, según las recomendaciones también») — firma electrónica simple abajo. Pilar y Lucía corroboran en el hub.
Dev	Sonda · limpieza 1 · compactar · limpieza 2 (v4) · cierre r27-b · Deploy Web (puertas) · arnés PG16 — todo el 26-sep.
Test	sql-columnas · parse PG16 · reglas puras en rojo/verde · e2e como anon (en el arnés y en producción) · arnés 5 escenarios para el cierre.
Deployment	Ejecutados por Juanjo en el SQL Editor: limpieza 1 (≈21:00), compactar ×2, limpieza 2 (4º intento ≈22:2x). Cierre r27-b: pendiente de Run (esta tanda).
Verification	Producción: recuentos (168 / 630.075 / 24 kB) + curl 400/23514. Humana pendiente: Pilar y Lucía abren el Piloto.
hub + Notion	Hub: este doc 53 + adjunto (po255). Notion: sin bloques desde el 8-sep → la verdad para ellas es el hub; pendiente anotado en CLAUDE.md.

Firmas

Persona	Rol	Conforme	Fecha
Juanjo Cebrián	Ingeniería y producto (D1-D9 y WL-1, carril técnico)	SÍ — firmado (abajo)	26-sep-2026
Pilar González	CEO · cumplimiento (corrobora: no había datos personales; borrado hecho)		
Lucía Cebrián	COO · diseño (corrobora: Piloto y cifra de la Home)		

Firma manuscrita de Juan José Cebrián González:



Firma electrónica simple · registro de integridad

Campo	Valor
Documento	UST-2026-09-26-18-incidente-waitlist-630k.pdf
Huella SHA-256 del contenido firmado (PDF sin firma)	cde220be7f57d4109de74270b3cd117b05fbcfe43ce57c0d5dd430960c0ee1a2
Marca de tiempo	2026-09-26 20:00:28 UTC · 2026-09-26 23:00:28 EEST
Firmante	Juan José Cebrián González · juanjose.cebrian@gmail.com · ingeniería y producto, NutriSync Collective
Decisiones firmadas	D1-D6 y D9 según las recomendaciones («UST-18, según las recomendaciones también», 26-sep): cuarentena con RLS sin políticas · tope de lote 20 · origen obligatorio para anon · ritmo global 200/10 min · CLUSTER en pasada propia · forense sin PII · arnés PG16 para todo fichero que mueva datos en masa. D7 = decisión previa de Juanjo (WL-1: «Limpiarlos, no se hace análisis, borrarlos» — cuarentena borrada en el cierre r27-b tras aserciones de recuento y PII) · D8 = NO (sin análisis de Logs). AS-BUILT: limpieza 1 · compactar ×2 · limpieza 2 (4º intento) · guardas probadas en producción (POST anon → 400/23514) · 168 vivas · heap 24 kB.
Medio	Firma manuscrita digitalizada aportada por Juanjo en el chat de trabajo + decisiones escritas por él. Firma electrónica SIMPLE (identidad + intención + integridad por huella y marca de tiempo).
Anclaje	La copia firmada y su huella SHA-256 (fichero .sha256 junto al PDF) quedan en el hub (documentation/ust) y en git.